Building Production Go Tools with AI-Assisted Development: A Case Study in ssql and autocli

Conference Paper Draft - Go Programming Conference

Ross Cartlidge, December 2025

Abstract

This paper presents a comprehensive case study of building two production-grade Go libraries—ssql (a stream processing library with SQL-style API) and autocli (a CLI framework with intelligent completion)—using AI-assisted development over a multi-month collaboration. We demonstrate methodologies for maintaining code quality, readability, and architectural coherence when working with Large Language Models (LLMs) as coding partners. The combined output includes 27,000+ lines of Go code, 40,000+ lines of documentation, and a self-generating CLI that can produce standalone Go programs 2.6x faster than interpreted execution. Key findings include the critical role of CLAUDE.md files as "AI memory," the importance of testing for preventing AI-introduced regressions, and patterns for keeping generated code readable and maintainable.

---

1. Introduction

1.1 The Challenge

Modern software development increasingly involves AI coding assistants. While these tools dramatically accelerate initial development, they introduce unique challenges:

- **Context loss**: LLMs have limited context windows and no persistent memory
- **Hallucination**: AI may generate plausible but incorrect API patterns
- **Code sprawl**: Rapid generation can lead to inconsistent, hard-to-maintain code
- **Regression risk**: AI changes can break previously working features

1.2 Our Approach

Over several months, we developed two interconnected Go libraries using Claude (Anthropic's LLM) as a primary coding partner:

1. **ssql** (v4.0.0) - A stream processing library with:
   - SQL-style API using Go 1.23+ iterators
   - Command-line tool for data processing pipelines
   - Self-generating code that compiles CLI commands to standalone Go programs
2. **autocli** (v4.3.3) - A CLI framework providing:
   - Fluent builder API for command construction
   - Intelligent bash completion with field-aware suggestions
   - Multi-argument flags with clause-based parsing

This paper documents the methodologies, problems solved, and lessons learned.

---

2. Project Statistics

2.1 Code Volume

Component	Lines of Go	Lines of Tests	Lines of Documentation
ssql core library	6,856	5,357	-
ssql CLI commands	5,804	-	-
ssql CLI lib/runtime	1,286	-	-
ssql CLI main	1,130	-	-
ssql examples	6,723	-	-
ssql documentation	-	-	29,140
autocli core	4,772	948	-
autocli examples	1,092	-	-
autocli documentation	-	-	10,569
Total	**27,663**	**6,305**	**39,709**

2.2 Version History

- **ssql**: 56 releases (v1.0.0 to v4.0.0)
- **autocli**: 17 releases (v1.0.0 to v4.3.3)
- **Major rewrites**: 4 (module path changes v2 to v3 to v4)
- **Breaking changes**: Documented and migrated cleanly each time

---

3. The CLAUDE.md Methodology

3.1 Problem: AI Has No Memory

LLMs process each conversation independently. Without persistent context, the AI: - Forgets architectural decisions from previous sessions - Regenerates code using outdated patterns - Doesn't know about custom APIs or conventions

3.2 Solution: CLAUDE.md as AI Memory

We created CLAUDE.md files in each repository—comprehensive documents that provide:

1. **Architectural decisions** with rationale
2. **API conventions** and naming patterns
3. **Anti-patterns** to avoid
4. **Code examples** of correct usage
5. **Development principles** learned from mistakes

ssql CLAUDE.md excerpt (1,407 lines):

```
## Development Principles (CRITICAL)

### If It's Not Tested, It Will Break

Features without tests will eventually be removed or broken during refactoring.
```

```
This was learned the hard way when field/value completion was accidentally
removed in v3.2.0 during a refactor. The feature worked, but had no test
coverage, so when code was reorganized the completion configuration was lost.

**Rules:**
- Add tests for any feature you want to keep
- Tests act as documentation of expected behavior
- Tests catch accidental removal during refactoring
- Don't assume "obvious" features will survive refactoring
```

3.3 Impact

With CLAUDE.md: - AI immediately understands project conventions - Consistent code style across sessions - Historical mistakes aren't repeated - New features align with existing architecture

Key insight: CLAUDE.md is both documentation for humans AND programming for the AI. Write it for both audiences.

---

4. Keeping Generated Code Readable

4.1 The Problem

AI can generate working but unreadable code:

```go
// BAD: AI-generated inline complexity
records := ssql.ReadCSV("data.csv")
result := ssql.LookupJoin(ssql.ReadCSV("lookup.csv"), []ssql.LookupClause{
    {LeftField: "a_kind", RightField: "kind", FieldRenames: map[string]string{"kind_name": "a_
    {LeftField: "z_kind", RightField: "kind", FieldRenames: map[string]string{"kind_name": "z_
})(records)
```

4.2 Solution: Helper Functions

Instead of generating inline complexity, we create helper functions in the library:

```go
// GOOD: Clean generated code using helper
records := ssql.ReadCSV("data.csv")
result := ssql.LookupJoin(ssql.ReadCSV("lookup.csv"), []ssql.LookupClause{
    ssql.Lookup("a_kind", "kind", "kind_name", "a_kind_name"),
    ssql.Lookup("z_kind", "kind", "kind_name", "z_kind_name"),
})(records)
```

Principle: Move complexity into the library; generated code should be self-documenting.

4.3 CLAUDE.md Documentation

```
## Generated Code Readability (CRITICAL)

**Rules for Code Generation:**
```

```
1. **Move complexity to helper functions** - Generated code should call
   helper functions in the ssql package, NOT inline complex logic
2. **Generated code should be self-documenting** - A reader should
   immediately understand what the pipeline does
3. **When adding new commands:**
   - First: Add helper function to ssql package
   - Then: Generate code that calls the helper
   - Test: Read the generated code - is the intent clear?
```

---

5. Testing Prevents AI Regressions

5.1 The Lesson Learned

In v3.2.0, AI-assisted refactoring accidentally removed field completion from CLI commands. The feature had worked for months, but without test coverage, it silently disappeared.

5.2 Solution: Test Everything Worth Keeping

We added comprehensive tests that verify feature presence:

```go
// TestFieldCompletionConfiguration verifies that all commands that accept
// field names have proper field completion configured (FieldsFromFlag)
// instead of NoCompleter. This test prevents regression where field
// completion is accidentally removed.
func TestFieldCompletionConfiguration(t *testing.T) {
    expectedFieldCompletion := map[string]map[string][]int{
        "where": {"-where": {0}},      // field is arg 0
        "update": {"-where": {0}, "-set": {0}, "-set-expr": {0}},
        "cast": {"-type": {0}},
        "join": {"-using": {0}, "-on": {0}},
        // ... all commands with field completion
    }

    // Verify each command has correct completers configured
    for subcmdName, flags := range expectedFieldCompletion {
        // ... verification logic
    }
}
```

5.3 CLAUDE.md Principle

The lesson was immediately documented in CLAUDE.md to prevent repetition:

> **If It's Not Tested, It Will Break**
>
> This test prevents regression where field completion is accidentally removed.

---

6. Code Generation: CLI to Compiled Go

6.1 Architecture

ssql CLI supports "self-generating pipelines" where commands emit Go code fragments:

```
# CLI execution (interpreted)
ssql from data.csv | ssql where -where age gt 25 | ssql to csv out.csv

# Code generation mode
export SSQLGO=1
ssql from data.csv | ssql where -where age gt 25 | ssql generate-go > program.go
go build -o program program.go
./program  # 2.6x faster than CLI
```

6.2 Fragment System

Each command can emit a JSON code fragment instead of executing:

```
// Code fragment types
type CodeFragment struct {
    Type    string   // "init", "stmt", "final", "func"
    Var     string   // Output variable name
    Input   string   // Input variable from previous command
    Code    string   // Go code for this operation
    Imports []string // Required imports
    Command string   // Original CLI command (for comments)
}
```

6.3 Performance Results

Real-world benchmark (enriching 10M+ records with multiple joins):

Method	Time	Speedup
CLI pipeline	2m 15s	baseline
Generated Go	52s	**2.6x faster**

The CLI overhead comes from: - Process spawning for each pipe stage - JSON serialization/deserialization between stages - Process substitution spawning subshells - Repeated file parsing

The generated Go code eliminates all of this—direct function calls, in-memory data structures, single-pass file reads.

6.4 Why It Matters

1. **Prototype in CLI** - Rapid iteration with immediate feedback
2. **Generate for production** - Compile to standalone binary
3. **Readable output** - Generated code is human-maintainable

---

7. autocli: Building the CLI Framework

7.1 Key Design Decisions

Fluent Builder API:

```
cmd := cf.NewCommand("ssql").
    Subcommand("where").
        Flag("-where").
            Arg("field").FieldsFromFlag("FILE").Done().
            Arg("operator").Completer(&StaticCompleter{...}).Done().
            Arg("value").FieldValuesFrom("FILE", "field").Done().
            Accumulate().Local().
        Done().
    Done()
```

Clause-based Parsing:

```
# Multiple conditions with OR logic using + separator
ssql where -where age gt 18 + -where status eq active
```

Intelligent Completion:

```
ssql where FILE users.csv -where <TAB>
# Shows: name, age, email, status (field names from file)

ssql where FILE users.csv -where status eq <TAB>
# Shows: active, pending, archived (actual data values!)
```

7.2 Features Developed

Over 17 releases, autocli evolved to include:

- **Nested subcommands** - git-style multi-level commands (remote add, container exec)
- **Three-level flag scoping** - Root global, subcommand global, and local (per-clause)
- **Field-aware completion** - Reads CSV/JSON files to suggest field names
- **Value completion** - Samples actual data to suggest filter values
- **Process substitution handling** - Special completion for `<(...)` paths
- **Multi-argument flags** - Per-argument completers with position awareness

7.3 CLAUDE.md Patterns

The autocli CLAUDE.md (496 lines) documents:

- Builder pattern implementation with Done() return types
- Three-level flag scoping and when to use each
- Completion script architecture and position calculation
- Common pitfalls (parser temporary commands, COMP_WORDS indexing)
- Builder interface over-application (a mistake we made and fixed)

---

8. Problem-Solving Case Studies

8.1 Case Study: Multi-Clause Joins (v4.0.0)

Problem: Users needed to enrich records from the same lookup file multiple times:

```
# Old (inefficient): Reads kind.csv twice
ssql join <(ssql from kind.csv | ssql rename ...) -on a_kind |
ssql join <(ssql from kind.csv | ssql rename ...) -on z_kind
```

Solution Process: 1. AI proposed multi-clause syntax with `-` separator 2. Human reviewed and requested performance optimization 3. AI implemented `LookupJoin` to read file once, build multiple indexes 4. Human requested helper functions for readable code generation 5. AI added `Lookup()` helper 6. Final result: 25% faster execution, cleaner syntax

```
# New (efficient): Reads kind.csv once
ssql join <(ssql from kind.csv) \
  -on a_kind kind -as kind_name a_kind_name \
  - \
  -on z_kind kind -as kind_name z_kind_name
```

8.2 Case Study: Process Substitution in Code Generation

Problem: Bash process substitution `<(...)` becomes `/dev/fd/N` paths that don't work in generated Go code.

Solution: 1. Detect non-regular files (pipes, FIFOs) via `os.Stat()` 2. Read subprocess fragments from the pipe 3. Generate Go functions that encapsulate the subprocess logic 4. Replace `/dev/fd/N` references with function calls

```
// Generated code for process substitution
func rightSource1() iter.Seq[ssql.Record] {
    records, _ := ssql.ReadCSV("kind.csv")
    return records
}

func main() {
    records, _ := ssql.ReadCSV("data.csv")
    result := ssql.LookupJoin(rightSource1(), clauses)(records)
}
```

8.3 Case Study: AI Self-Improvement Loop

Problem: LLMs hallucinate incorrect ssql APIs (combined GroupBy+Aggregate, wrong Count() parameters).

Solution: Created automated validation suite that: 1. Tests generated code against reference implementations 2. Identifies common AI mistakes 3. Updates AI prompt with anti-patterns 4. Re-validates to confirm improvement

The anti-patterns section was added to the AI code generation prompt:

> **CRITICAL ANTI-PATTERNS**

LLMs often hallucinate these WRONG APIs - DO NOT USE:

Wrong: Combined GroupBy + Aggregate API (doesn't exist!) **Correct:** Separate GroupBy and Aggregate calls

This created a feedback loop where AI mistakes improved the prompt, which reduced future mistakes.

8.4 Case Study: Record Encapsulation (v1.0.0)

Problem: The original `Record` type was `map[string]any`, which allowed invalid types to be inserted and made it hard to enforce type safety.

Solution Process: 1. AI proposed encapsulating Record as a struct with private fields 2. Human approved the breaking change for v1.0.0 3. AI implemented MutableRecord builder pattern 4. AI added GetOr(), GetT accessor methods 5. AI updated all 40+ example files 6. Human verified compile-time safety was enforced

```
// Before (v0.x): Any type could be stored
record["age"] = "not a number"  // Compiles but wrong

// After (v1.0+): Type-safe accessors
record := ssql.MakeMutableRecord().
    Int("age", int64(30)).
    Freeze()
age := ssql.GetOr(record, "age", int64(0))
```

---

9. Development Workflow

9.1 Human-AI Collaboration Pattern

The collaboration followed a consistent pattern across hundreds of development sessions:

Phase 1: Problem Definition (Human-Led) - Human describes the feature goal and business context - Human identifies constraints (backward compatibility, performance requirements) - Human provides examples of desired behavior

Phase 2: Design Exploration (AI-Led, Human-Reviewed) - AI proposes implementation approaches with trade-offs - AI identifies affected files and potential breaking changes - Human selects approach or requests alternatives - For complex features, AI writes design doc before coding

Phase 3: Implementation (AI-Led) - AI implements feature with tests - AI updates documentation alongside code - AI ensures all existing tests still pass

Phase 4: Review and Refinement (Human-Led) - Human reviews generated code for readability - Human runs tests and verifies behavior - Human requests specific changes ("make this more readable", "add error handling") - AI iterates until human approves

Phase 5: Knowledge Capture (Both) - AI updates CLAUDE.md with lessons learned - Human verifies CLAUDE.md captures key decisions - Both document breaking changes in CHANGELOG

9.2 Quality Gates

Before merging any AI-generated code:

- All existing tests pass
- New tests added for new features
- Code is readable (not over-abstracted)
- Documentation updated
- CLAUDE.md updated if patterns discovered
- Breaking changes documented with migration path

9.3 Version Control Discipline

- **Atomic commits**: Each feature/fix in one commit
- **Descriptive messages**: Include context for future AI sessions
- **Annotated tags**: Full release notes in tag messages
- **Clean main branch**: No local `replace` directives in go.mod
- **Semantic versioning**: Breaking changes get major version bumps

9.4 Communication Patterns

Effective prompts to the AI included:

- "Have you checked the code generation for the new feature?"
- "Are you making sure the generated code is simple by adding helpers to the ssql package?"
- "Are there tests for this new feature?"
- "Have the docs been updated?"
- "Is it pushed to GitHub?"

These prompts established a checklist that the AI learned to anticipate.

---

10. Lessons Learned

10.1 What Worked

1. **CLAUDE.md as AI memory** - Essential for multi-session consistency. The 1,407-line ssql CLAUDE.md became the authoritative source for project conventions.

2. **Test-driven features** - If it's not tested, AI will eventually break it. This was learned painfully with the v3.2.0 completion regression.

3. **Helper functions over inline code** - Keeps generated code readable and maintainable. The `Lookup()` helper is a perfect example.

4. **Incremental complexity** - Start simple, add features gradually. Each ssql release built on the previous one.

5. **Human review of architecture** - AI handles implementation, human guides design. AI is excellent at generating code but needs direction on structure.

6. **Anti-pattern documentation** - Prevents repeated AI mistakes. The AI code generation prompt's anti-patterns section was crucial.

7. **Comprehensive documentation** - 40,000+ lines of docs meant the AI always had context for generating correct code.

10.2 What Didn't Work

1. **Trusting AI "improvements"** - Always verify refactoring doesn't break features. AI eagerly refactors but doesn't always preserve behavior.
2. **Complex inline generation** - Generated code became unreadable. Had to retrofit helper functions.
3. **Missing context** - Without CLAUDE.md, AI made inconsistent choices between sessions.
4. **Skipping tests** - Led to silent regressions that were only discovered by users.
5. **Over-abstraction** - AI tends to over-engineer. Had to explicitly request simpler solutions.

10.3 Key Principles

AI-ASSISTED DEVELOPMENT PRINCIPLES

1. Document everything in CLAUDE.md - it's AI memory
2. Test everything you want to keep
3. Prefer compile-time type safety over runtime validation
4. Keep generated code simple and readable
5. Human reviews architecture, AI implements
6. Atomic changes with comprehensive commit messages
7. Update documentation alongside code

---

11. Future Work

11.1 Planned Enhancements

- **GPU acceleration** for large dataset processing using Go's upcoming GPU support
- **Schema-aware records** with compile-time field validation
- **Streaming aggregations** for unbounded data sources
- **WASM compilation** for browser-based data processing

11.2 AI Collaboration Improvements

- **Automated CLAUDE.md updates** - Parse test failures to auto-generate anti-patterns
- **Pattern library** - Curated collection of successful AI prompts
- **Regression detection** - Compare generated code across versions for drift
- **Multi-model collaboration** - Use different models for different tasks (design vs implementation)

11.3 Open Questions

- How do we handle AI context limits as CLAUDE.md grows?
- Can we train custom models on our codebase?
- What's the right granularity for AI-generated commits?

---

12. Conclusion

Building ssql and autocli with AI assistance demonstrates that LLMs can be effective partners for production software development. The key is treating the AI as a capable but forgetful collaborator who needs clear documentation (CLAUDE.md), quality gates (tests), and architectural guidance (human review).

The results speak for themselves: - **27,000+ lines** of production Go code - **40,000+ lines** of documentation - **2.6x performance improvement** from code generation - **Clean API evolution** through 4 major versions - **73 releases** over several months

The methodology presented—CLAUDE.md files, comprehensive testing, helper functions for generated code, and iterative human-AI collaboration—provides a template for other teams exploring AI-assisted development.

The collaboration was genuinely productive. The AI handled tedious tasks (updating 40+ example files for API changes), caught issues the human missed (incomplete error handling), and proposed creative solutions (the code fragment system for generation). The human provided direction, caught architectural issues, and ensured code quality.

This is not AI replacing developers—it's AI augmenting them. The human-AI team produced more, faster, with higher quality than either could alone.

Appendix A: Code Statistics Summary

COMBINED PROJECT STATISTICS

Category	Lines
Go source code	27,663
Test code	6,305
Documentation	39,709
Examples	7,815
Total	**81,492**

Metric	Count
ssql releases	56
autocli releases	17
Total releases	**73**

Appendix B: Repository Links

- **ssql**: github.com/rosscartlidge/ssql
- **autocli**: github.com/rosscartlidge/autocli

Appendix C: Sample CLAUDE.md Structure

A well-structured CLAUDE.md includes:

1. **Project Overview** - What the project does
2. **Repository Hygiene** - Where files should go
3. **Development Principles** - Hard-won lessons
4. **API Conventions** - Naming patterns, type rules
5. **CLI Design Patterns** - For tools with CLIs
6. **Code Generation Rules** - If applicable
7. **Testing Strategy** - What to test and how
8. **Common Pitfalls** - Mistakes to avoid
9. **Version History** - Breaking changes and migrations
10. **References** - Links to detailed docs

Appendix D: Recommended Reading

- "Prompt Engineering Guide" - Best practices for AI prompts
- "Working Effectively with Legacy Code" - Patterns for safe refactoring
- "Release It!" - Production-ready software patterns
- Go 1.23 Release Notes - Iterator (iter.Seq) documentation

Paper prepared for Go Programming Conference 2026

The ssql and autocli projects are open source and available on GitHub.